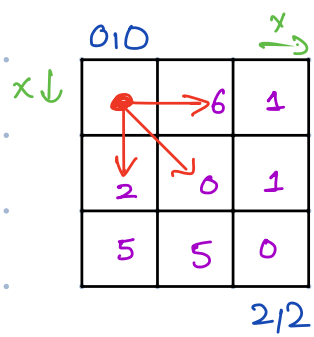


A navigation system directs a rover across an $N \times M$ grid. The rover starts at $(0, 0)$ and must reach $(N - 1, M - 1)$. It can move to $(x + 1, y)$, $(x, y + 1)$, or $(x + 1, y + 1)$ in a single step. Each coordinate contains a mineral deposit of varying mass. The rover collects the mass from each coordinate it enters. State the local selection criteria to maximize collected mass. Construct a 3×3 grid with specific mass values that proves your local selection criteria results in a sub-optimal total mass.



Problem 2

An automated teller dispenses currency using only 1-unit, 10-unit, and 15-unit bills. You must dispense exactly K units using the absolute minimum number of bills. State the algorithm to select the bills. Construct a fail case with a specific value for K that proves this algorithm does not yield the minimum number of bills.

$K = 20$

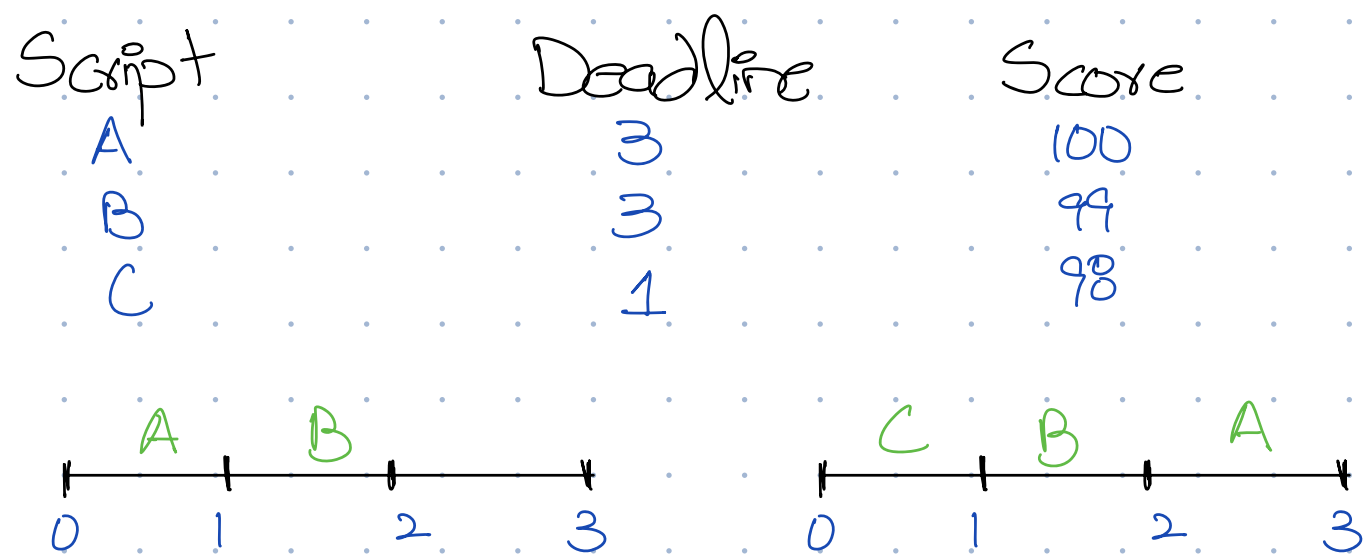
Greedy way = 15, 1, 1, 1, 1, 1

Optimal selection = 10, 10

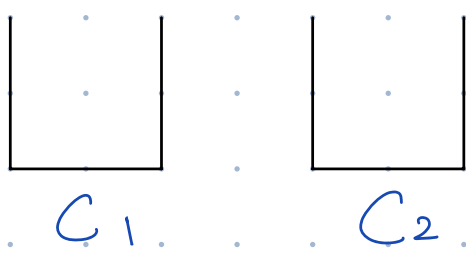
A processor must execute N distinct tasks. Each task requires continuous processing from a specific, fixed start timestamp to a specific, fixed end timestamp. The processor can only handle one task at a time. State the algorithmic sorting and selection logic required to execute the maximum possible number of tasks. Construct a specific list of 4 tasks (with start and end timestamps) that proves sorting the tasks by shortest processing duration fails to maximize the number of executed tasks.

Task	Start	End	Duration
A	1	4	3
B	3	5	2
C	4	7	3
D	8	9	1

A server has N scheduled maintenance scripts. Every script takes exactly 1 minute to run. Each script has a deadline (in minutes from system boot) and a priority score. If a script finishes before or exactly on its deadline, the system gains the priority score. State the logic to sort and assign scripts to specific 1-minute execution slots to maximize the total score. Construct a specific list of 3 scripts (with deadlines and scores) that proves assigning a selected script to the earliest available time slot, rather than your proposed slot, forces a sub-optimal final score.



- Description:** A cargo vessel has two identical side-by-side cargo holds (Port and Starboard). You have N shipping containers of various distinct weights. To ensure maximum stability on the water, you must divide the containers between the two holds so that the absolute difference in total weight between the Port and Starboard holds is as close to zero as possible.



Cargo	weight	Objective: $ C_1 - C_2 \approx 0$
1	10	
2	20	
3	5	
4	3	

```
C1.add(2)
for item_w in sorted(cargos, Descending, skip-first):
    curDiff = |C1-w - C2-w|
    i = argmin[|item_w + C1-w - C2-w|, |item_w + C2-w - C1-w|]
    C[i].add(item_w)
```

This fails for: 5, 4, 3, 2, 2

$(4+5) - 0, (4+0) - 5$
 $2 = (9, 1)$

$(3+5) - 4, (3+4) - 5$
 $4, 2$

$(4+2) - 7, (7+2) - 4$
 $1, 4$

$(7+2) - 7, (7+2) - 7$
 $2, 2$

$C_1 = 5, 2, 2$ NOT optimal

$$C_2 = 4, 3$$

$$C_1 = 5, 3$$

actual optimal

$$C_2 = 4, 2, 2$$

So, greedy fails

A straight pipeline is M meters long. There are n critical leak-risk points located at specific coordinate markers along the pipeline. You must deploy sensor devices to monitor these points. Each sensor device can monitor exactly a continuous K -meter segment of the pipeline. Formulate an algorithm to determine the optimal placement of the minimum number of sensors required to cover all n risk points.

```
sort locations in ascending order
prev_covered_boundary = -infinity

for each x_i in locations:
    if x_i > prev_covered_boundary:
        place sensor at x_i
        prev_covered_boundary = x_i + K
```